

Doxygon

Reference Manual

Comment-Driven Documentation



目次

1. はじめに	1
1.1. 用語について	1
1.2. Doxygen とは	1
1.3. Doxygen との違い	1
1.4. 対応プログラミング言語	2
2. Doxygen の基本操作	3
2.1. Doxygen の入手先	3
2.2. 実行環境の設定	4
2.3. パラメータの設定	4
2.4. Doxygen の実行	5
2.5. 成果物の活用方法	5
3. Doxygen コメントの記述方法	6
3.1. コメントブロックの構造	6
3.2. コメントブロックの記述ルール	6
3.3. 言語別コメントブロック	7
3.4. AsciiDoc との併用	9
4. Doxygen コマンド	11
4.1. Doxygen コマンドとは	11
4.2. 利用できる Doxygen コマンド	11
4.3. コマンドの構文	11
4.3.1. @attention コマンド	11
4.3.2. @author コマンド	12
4.3.3. @brief コマンド	13
4.3.4. @class コマンド	13
4.3.5. @define コマンド	14
4.3.6. @deprecated コマンド	14
4.3.7. @details コマンド	15
4.3.8. @enum コマンド	15
4.3.9. @figure コマンド	16
4.3.10. @file コマンド	17
4.3.11. @fn コマンド	17
4.3.12. @important コマンド	18
4.3.13. @interface コマンド	19
4.3.14. @note コマンド	19
4.3.15. @par コマンド	20
4.3.16. @param コマンド	20
4.3.17. @return コマンド	21
4.3.18. @struct コマンド	21
4.3.19. @table コマンド	22
4.3.20. @tip コマンド	23
4.3.21. @todo コマンド	23
4.3.22. @type コマンド	24
4.3.23. @union コマンド	24
4.3.24. @var コマンド	25

4.3.25. @warningコマンド	25
5. Doxygen で使用する AsciiDoc 記法	27
5.1. 段落と改行	27
5.2. 文字の装飾	27
5.3. 見出し	28
5.4. 箇条書き	28
5.5. ラベル付きリスト	29
5.6. ブロック	30
5.7. アドモニション	30
5.8. 画像	31
5.9. テーブル	32
5.10. ソースコード	33
5.11. コメント	35
5.12. DoxygenコマンドとAsciiDoc記法の対比	36

1. はじめに

Doxygen Reference Manual は、Doxygen が対応するコメントブロック、Doxygen コマンド、および Doxygen 独自仕様をまとめたリファレンスマニュアルです。
本書は「最初から順番に読む」ことを目的とした解説書ではありません。必要な項目を必要なときに参照できるように、リファレンスとして構成しています。

Doxygen の設計思想や実践的な運用方法については、将来的に Doxygen Playbook としてまとめる予定です。

1.1. 用語について

本書では、以下の用語を使用します。

コメントブロック

コメント開始記号とコメント終端記号で囲まれた領域です。
Doxygen は、コメントブロックを解析対象とします。

Doxygen コマンド

コメントブロック内で使用するコマンドです。
Doxygen と同じ名称を持つコマンド、および Doxygen 独自コマンドを含みます。
Doxygen と仕様が異なるコマンドについては、「[Doxygen コマンド](#)」で説明します。

成果物

Doxygen が生成する AsciiDoc ファイルです。

コメントドリブン

コメントドリブン（Comment-Driven Development）は、実装する処理の内容や構成をコメントとして記述し、そのコメントを基にプログラムを実装していく開発手法です。
コメントを先に記述することで、実装前に処理の流れや設計内容を整理できるため、設計と実装の整合性を保ちやすくなります。

1.2. Doxygen とは

Doxygen は、コメントブロックから AsciiDoc 形式の成果物を生成するドキュメントジェネレータです。
コメントは、ソースコードを説明するための付加情報ではなく、**設計思想を伝えるための重要な情報**と考えられています。



AsciiDoc とは？

AsciiDoc は、技術文書の作成を目的としたプレーンテキストの軽量マークアップ言語です。

1.3. Doxygen との違い

Doxygen は、ソースコードから API リファレンスを生成するツールとして広く利用されています。
一方、Doxygen は、ソースコードを起点として成果物を生成するのではなく、コメントを起点として成果物を生成します。
つまり、**ソースコードが存在しない設計段階から**成果物を生成することができます。

また、Doxygen の主な成果物は、HTML、LaTeX、xml 形式であるため、容易に編集することができません。

Doxygon の成果物は AsciiDoc 形式を採用しているので、テキストエディタで自由に追加、編集することができます。

Design documents start with comments, not source code.

Doxygon では Doxygen のコメント記法を採用しながらも、AsciiDoc を前提とした記述を可能とします。

1.4. 対応プログラミング言語

現在、Doxygon は以下のプログラミング言語に対応しています。

- **VBA**
- C
- C++
- Java
- JavaScript
- TypeScript
- Python

コメントブロックの記述方法は言語ごとに異なりますが、Doxygen コマンドの記述方法はすべて共通です。

本書が、Doxygon を利用した設計ドキュメントを継続的に運用、保守するためのリファレンスとして役立つことを願っています。

2. Doxygon の基本操作

この章では、Doxygon を利用して成果物を生成するまでの基本的な手順について説明します。Doxygon は、Windows および Linux（将来）向けの実行ファイル（バイナリ）を提供しています。実行ファイルを使用することにより、Doxygon を開発・カスタマイズする場合を除き、Python 環境を構築する必要はありません。

2.1. Doxygon の入手先

Doxygon は [GitHub Release](#) からダウンロードできます。配布ファイルの内容を [表 2.1](#) に示します。

表 2.1 配布ファイルの構成

配布物	用途
Windows	Windows 用実行ファイル
Source Code	ソースコード

2.2. 実行環境の設定

Doxygen は、設定ファイル (config.toml) を読み込み、指定されたフォルダー内のソースコードを解析し、AsciiDoc ファイルを生成します。

なお、ダウンロードした zip ファイルは、Doxygen 本体を含む配布パッケージのため、サンプル用のフォルダー構成となっています。実際に使用する場合は、[例 2.1](#)に示す構成例を元にフォルダーを構成して下さい。

例 2.1 Doxygen 実行環境フォルダー構成

```
/path/to/project/  
├─ config/  
│   └─ command.toml ①  
├─ examples/ ②  
│   └─ adoc-template  
│       ├── attributes  
│       │   └─ initChartNo.adoc ③  
│       └─ main.adoc ④  
├─ input/ ⑤  
│   ├── sample.c  
│   └─ ...  
├─ output/ ⑥  
│   ├── src/  
│   │   ├── sample.c  
│   │   └─ ...  
│   ├── sample.c.adoc  
│   └─ ...  
├─ config.toml ⑦  
└─ doxygen.exe ⑧
```

- ① : Doxygen コマンド定義ファイル (変更禁止)
- ② : AsciiDoc 用サンプル
- ③ : 図表番号初期化マクロ
- ④ : AsciiDoc メインファイル
- ⑤ : 解析対象ソースファイル格納フォルダー
- ⑥ : AsciiDoc ファイル出力先フォルダー
- ⑦ : Doxygen パラメータ設定ファイル
- ⑧ : Doxygen 実行ファイル

2.3. パラメータの設定

Doxygen の動作に関するパラメータは、config.toml で設定します。最低限必要な設定は以下の項目です。

```
[paths]
input_dir = "./input" ①
output_dir = "./output" ②
```

①：解析対象ソースファイル格納フォルダー

②：AsciiDoc ファイル出力先フォルダー

2.4. Doxygen の実行

設定ファイルを配置したディレクトリで Doxygen を実行します。

```
> doxygen
```

または

```
> doxygen config.toml
```

と実行すると、設定に従って AsciiDoc ファイルが生成されますので、そのままテキストファイルとして閲覧、編集することができます。

2.5. 成果物の活用方法

生成された AsciiDoc ファイルは、必要に応じて Asciidoctor などのツールを用いて、HTML/PDF に変換してください。

なお、簡易的にブラウザーで確認したい場合は、ブラウザー専用の Asciidoctor 拡張機能を利用してください。

Chrome をご利用のかた

[Chrome ウェブストア](#)からインストールしてください。

Edge をご利用のかた

[Edge アドオンストア](#)からインストールしてください。

3. Doxygen コメントの記述方法

この章では、コメントブロックの記述方法について説明します。
コメントブロックには、Doxygen コマンドと AsciiDoc を組み合わせて設計情報を記述します。

3.1. コメントブロックの構造

コメントブロックの構造をイメージ化したものを図 3.1 に示します。
コメントブロックは AsciiDoc を基本構造としており、Doxygen コマンドは AsciiDoc の一部として記述します。
なお、AsciiDoc は省略することもできます。

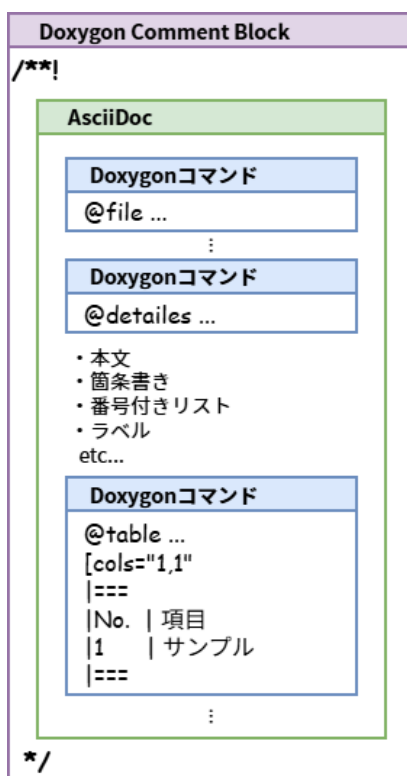


図 3.1 コメントブロック構造図



コメントブロックの中は「Doxygen コマンド」と「AsciiDoc」が共存する領域です。

3.2. コメントブロックの記述ルール

Doxygen コメントブロックでは、次のルールに従って記述してください。

- Doxygen コマンドは行頭から記述します。
- Doxygen コマンドは 1 行に 1 つとします。
- Doxygen コマンド以外は AsciiDoc として扱われます。
- 空行は AsciiDoc の段落区切りとして扱われます。
- AsciiDoc の各種ブロックを利用できます。

例えば、コメントブロック内に以下のような記述があると、

- ***** は箇条書き
- **==** は見出し
- **NOTE:** は注記

のように AsciiDoc の記法として解釈されます。

3.3. 言語別コメントブロック

表 3.1 に言語別のコメント開始／終端、およびインラインコメントの構文を示します。
なお、Python のインラインコメントは対応しておりません。

表 3.1 言語別コメントブロック

言語	コメント開始記号	コメント終端記号	インラインコメント
VBA	'/**!	*/	'/**<
C/C++	/**!	*/	/**!<
Java	/**!	*/	/**!<
JavaScript	/**!	*/	/**!<
TypeScript	/**!	*/	/**!<
Python	"""!	"""	(対象外)

また、複数行に渡るコメントを記述する場合、先頭に ***** があると「箇条書き」として認識されますので、ご注意ください。

ただし、VBA には複数行コメントという概念がありませんので、すべてのコメント行の先頭に **'**（シングルクォーテーション）が必要です。

VBA のコメントの正しい書き方を例 3.1 に、誤った書き方を例 3.2 に示します。

同様に C/C++ 言語系統のコメントの書き方を例 3.3、例 3.4 に、Python のコメントの書き方を例 3.5、例 3.6 にそれぞれ示します。



コメント開始記号からコメント終端記号までの間は、AsciiDoc 記法が優先されます。

例 3.1 VBA コメントの書き方（正）

```
' /**!  
'  
' VBA では、このような書き方をします。  
'  
' */
```

例 3.2 VBA コメントの書き方（誤）

```
' /**!
```

```
' *  
' * VBA では、このような書き方をしてはいけません。  
' *  
' */
```

例 3.3 C/C++ 言語系統 コメントの書き方（正）

```
/**!  
  
C/C++ 言語系統では、このような書き方をします。  
  
*/
```

例 3.4 C/C++ 言語系統 コメントの書き方（誤）

```
/**!  
 *  
 * C/C++ 言語系統では、このような書き方をしてはいけません。  
 *  
 */
```

例 3.5 Python コメントの書き方（正）

```
"""!  
  
Python では、このような書き方をします。  
  
"""
```

例 3.6 Python のコメントの書き方（誤）

```
###!  
#  
# Python では、このような書き方をしてはいけません。
```

```
#  
###
```

3.4. AsciiDoc との併用

コメントブロック内では、通常の AsciiDoc と同様に各種ブロックやマクロを利用できます。また、AsciiDoc の拡張機能である PlantUML によるシーケンス図やクラス図を埋め込むこともできます。

図 3.2 にコメントブロックの一例を示します。Doxygen コマンドと AsciiDoc 記法が共存していることがわかります。

図 3.2 コメントブロック例

```
/**! ①  
@brief サンプル関数 ②  
  
関数の概要を記述します。  
  
== 処理概要 ③  
  
* 入力値を検証します。 ④  
* 条件判定を行います。  
* 結果を返します。  
  
NOTE: エラー時は false を返します。 ⑤  
  
@param value 入力値 ②  
@return true: 成功 / false: 失敗 ②  
  
@figure fig1 PlantUML の記述例 ⑥  
[plantuml] ⑦  
----  
Bob -> Alice  
----  
*/ ⑧
```

- ① : コメントブロック開始記号 (C 言語)
- ② : Doxygen コマンド
- ③ : AsciiDoc 記法 (見出し)

- ④：AsciiDoc 記法（箇条書き）
- ⑤：AsciiDoc 記法（アドモニション）
- ⑥：Doxygen コマンド
- ⑦：PlantUML 記法
- ⑧：コメントブロック終端記号（C 言語）

各コマンドの詳細については、第4章「Doxygen コマンド」で説明します。

4. Doxygenコマンド

4.1. Doxygenコマンドとは

Doxygen は、基本的に Doxygen コマンドに準拠しています。
これは、Doxygen のコメント記法に慣れたユーザーが、そのまま利用できることを目的としています。

一方、Doxygen は、この考え方をさらに発展させ、コメントから設計ドキュメントを生成し、その成果物を継続的に保守・活用することを目的としています。
そのため、設計ドキュメントの構造そのものをコメントで表現できるように設計しました。

この考え方を実現するため、Doxygen では一部の Doxygen コマンドを再定義するとともに、独自コマンドを追加しています。

4.2. 利用できるDoxygenコマンド

Doxygen で利用できるコマンドと Doxygen との互換性について表 4.1 に示します。
Doxygen を利用した経験がある場合は、「Doxygen 互換」の欄も合わせて確認してください。

表 4.1 利用できる Doxygen コマンド

コマンド	Doxygen 互換	コマンド	Doxygen 互換	コマンド	Doxygen 互換
@attention	○	@file	△	@table	×
@author	△	@fn	△	@tip	×
@brief	○	@important	○	@todo	○
@class	○	@interface	○	@type	×
@define	×	@note	○	@union	○
@deprecated	○	@par	○	@var	△
@details	○	@param	○	@warning	○
@enum	○	@return	○		
@figure	×	@struct	○		

- ：Doxygen と同じ使い方ができます。
△：Doxygen と同名のコマンドですが、Doxygen では使い方が異なります。
×：Doxygen には存在しない Doxygen 独自コマンドです。

4.3. コマンドの構文

本章では、コマンドの構文を次の記号で表します。

- 【】：必須項目です。
- < >：任意項目です。

4.3.1. @attentionコマンド

注意事項を定義します。

構文

```
@attention 【注意事項】
```

記述例

```
/**!  
@attention この関数を呼び出す前に initialize() を実行してください。  
*/
```

出力例

確認ポイント
この関数を呼び出す前に initialize() を実行してください。

4.3.2. @authorコマンド

担当者を定義します。

構文

```
@author 【担当者名】 <メールアドレス>
```

記述例

```
/**!  
@author Geddy Lee <geddy.lee@rush.com>  
@author Alex Lifeson alex.lifeson@rush.com  
@author Neil Peart  
*/
```

出力例

担当者
Geddy Lee <geddy.lee@rush.com>
Alex Lifeson <alex.lifeson@rush.com>
Neil Peart

備考

- 同一コメントブロック内に定義された **@author** は1つの段落として結合します。
- メールアドレスは、電子メールの標準規格に基づき `< >` で括弧することを推奨します。（省略可）

4.3.3. @briefコマンド

処理概要を定義します。

構文

```
@brief 【処理概要】
```

記述例

```
/**!  
@brief 四則演算を行います  
*/
```

出力例

```
処理概要  
四則演算を行います
```

4.3.4. @classコマンド

クラス名を定義します。

構文

```
@class 【クラス名】 <クラス和名>
```

記述例

```
/**!  
@class ConfigReport 設定レポート生成クラス  
*/
```

出力例

```
1.1.1 クラス詳解
```

1.1.1.1. ConfigReport 設定レポート生成クラス

備考

- 定義されたクラスは、クラス一覧に表示されます。

4.3.5. @defineコマンド

マクロ定義名を定義します。

構文

```
@define 【マクロ定義名】 <マクロ定義和名>
```

記述例

```
/**!< @define SENSOR_BUFFER_SIZE センサー値保持件数 */
```

出力例

マクロ

- SENSOR_BUFFER_SIZE : センサー値保持件数

備考

- 定義されたマクロ定義名は、マクロ一覧に表示されます。
- Doxygen では、"@def" コマンドに相当します。

4.3.6. @deprecatedコマンド

非推奨項目を定義します。

構文

```
@deprecated 【非推奨項目】
```

記述例

```
/**!  
@deprecated この関数は v2.0 以降非推奨です。代わりに getUsernameNew()  
を使用してください。  
*/
```

出力例

非推奨

この関数は v2.0 以降非推奨です。代わりに `getUserNameNew()` を使用してください。

4.3.7. @details コマンド

詳細内容を定義します。

構文

```
@details 【詳細内容】
```

記述例

```
/**!  
@details  
 * ユーザーIDが既に存在する場合は追加されず、false を返します。  
 * 名前は空文字列を許可しません。  
 */
```

出力例

処理詳細

- ユーザーIDが既に存在する場合は追加されず、false を返します。
- 名前は空文字列を許可しません。

備考

- 箇条書きにする場合は、文章の先頭に * を付加します。

4.3.8. @enum コマンド

列挙型名を定義します。

構文

```
@enum 【列挙型名】 <列挙型和名>
```

記述例

```
/**!  
@enum PrimaryStatus 優先稼働状態を表す列挙体  
*/
```

出力例:

列挙体

- Primarystate: 優先稼働状態を表す列挙体

4.3.9. @figureコマンド

図のタイトルを定義します。

構文

```
@figure 【図のタイトル】
```

記述例

```
/**!  
@figure ゴキシドンのイメージ  
image:../images/Goxydon_s.png[width="20%",align="center"]  
*/
```

出力例



図 4.1 ゴキシドンのイメージ

備考

@figure コマンド単体では使用することができません。

@figure コマンドのあとに **image** マクロによる画像ファイルの指定、または Graphviz, PlantUML などの図形描画ツールによるソースコードを記述する必要があります。

image マクロについては、「[画像](#)」を参照してください。

図番号の自動採番には、`examples/adoc-template/attributes` フォルダーの下にある `initChartNo.adoc` を使用します。
使い方の例は、`examples/adoc-template/main.adoc` を参照してください。

4.3.10. @fileコマンド

ファイル名を定義します。

構文

```
@file 【ファイル名】 <ファイル和名>
```

記述例

```
/**!  
@file helloworld.c こんにちは、世界  
*/
```

出力例

1.1 helloworld.c ファイル
[こんにちは、世界](#)

解説

ソースコードが存在しない設計フェーズにおいても `@file` コマンドを起点として設計ドキュメントを作成します。`@file` コマンドは、**1 ファイルに必ず 1 つ定義**してください。
また、和名をクリックすると当該ソースコードへジャンプします。

備考

- `@file` が複数記述された場合、2つめ以降は `[DUPLICATE]` を示す警告が表示されます。
- `@file` に指定したファイル名は、実ファイル名より優先されます。

4.3.11. @fnコマンド

関数名を定義します。

構文

```
@fn 【関数名】 <関数和名>
```

記述例

```
/**!  
@fn add 2 つの整数を加算します。  
*/
```

出力例

1.1.1. 関数詳解

1.1.1. add
2 つの整数を加算します。

コーリングシーケンス

```
int add(int a, int b)
```

備考

- 関数名に引数は記述しません。

4.3.12. @important コマンド

重要事項を定義します。

構文

```
@important 【重要事項】
```

記述例

```
/**!  
@important "@file" コマンドは 1 ファイルに 1 つだけ定義してください。  
*/
```

出力例



"@file" コマンドは 1 ファイルに 1 つだけ定義してください。

備考

- AsciiDoc のデフォルトアイコンを使用します。
- Doxygen コマンド名を記述するには、""（ダブルクォーテーション）で括る必要があります。

4.3.13. @interfaceコマンド

インターフェイス名を定義します。

構文

```
@interface 【インターフェイス名】 <インターフェイス和名>
```

記述例

```
/**!  
@interface SensorInterface センサー制御インターフェイス  
*/
```

出力例

インターフェイス

- SensorInterface : センサー制御インターフェイス

備考

- 定義されたインターフェイス名は、インターフェイス一覧に表示されます。

4.3.14. @noteコマンド

メモを定義します。

構文

```
@note 【メモ】
```

記述例

```
/**!  
@note "@note" コマンドの例です。  
*/
```

出力例



"@note" コマンドの例です。

備考

- Doxygon コマンド名を記述するには、""（ダブルクォーテーション）で括る必要があります。

4.3.15. @par コマンド

ユーザー定義の段落名を定義します。

構文

```
@par 【ユーザー定義の段落名】
```

記述例

```
/**!  
@par 備考  
ユーザーにて「備考」という段落名を定義しました。  
*/
```

出力例

```
備考  
ユーザーにて「備考」という段落名を定義しました。
```

4.3.16. @param コマンド

引数を定義します。

構文

```
@param <[in] | [in,out] | [out]> 【引数名】 <引数和名>
```

記述例

```
/**!  
@param [in] age ユーザーの年齢  
*/
```

出力例

```
引数  
[in] age
```

ユーザーの年齢

備考

- 関数内に定義された引数名は、引数一覧に表示されます。

4.3.17. @returnコマンド

戻り値名を定義します。

構文

```
@return 【戻り値名】 <戻り値和名>
```

記述例

```
/**!  
@return errorflag エラーフラグ  
*/
```

出力例

```
戻り値  
errorflag エラーフラグ
```

備考

- 関数内に定義された戻り値は、戻り値一覧に表示されます。

4.3.18. @structコマンド

構造体名を定義します。

構文

```
@struct 【構造体名】 <構造体和名>
```

記述例

```
/**!  
@struct SensorBuffer センサー値保存バッファ  
*/
```

出力例

構造体

- SensorBuffer : センサー値保存バッファ

備考

- 定義された構造体は、構造体一覧に表示されます。

4.3.19. @tableコマンド

テーブル名を定義します。

構文

```
@table   【テーブル名】
```

記述例

```
/**!  
@table Doxygon タイトル一覧  
[cols="1,2", options="header", width="90%", role="center"]  
|===  
  
^.^| 話数  
^.^| タイトル  
  
| 第壱話  
| 使徒、襲来  
  
| 第弐話  
| 見知らぬ、天井  
  
| 第参話  
| 鳴らない、電話  
  
|===  
*/
```

出力例

表 4.2 タイトル一覧

話数	タイトル
第壱話	使徒、襲来
第弐話	見知らぬ、天井
第参話	鳴らない、電話

備考

@table コマンド単体では使用することができません。

@table コマンドのあとに AsciiDoc 記法によるテーブルを記述する必要があります。
テーブルの作成方法については、「[テーブル](#)」を参照してください。

4.3.20. @tipコマンド

ヒントを定義します。

構文

```
@tip 【ヒント】
```

記述例

```
/**!  
@tip 効率的に記述するには、ショートカットキーを活用するのがおすすめです。  
*/
```

出力例



効率的に記述するには、ショートカットキーを活用するのがおすすめです。

4.3.21. @todoコマンド

"TODO" を定義します。

構文

```
@todo 【TODOの内容】
```

記述例

```
/**!  
@todo 今日できることは、明日やる。  
*/
```

出力例

TODO
今日できることは、明日やる。

4.3.22. @type コマンド

型定義名を定義します。

構文

```
@type 【型定義名】 <型定義和名>
```

記述例

```
typedef long BlockType; /**!< @type BlockType ブロックコメント用型定義 */
```

出力例

型定義

- BlockType: ブロックコメント用型定義

備考

- 「記述例」はインラインコメントを使用した場合のものです。
- 定義された型定義は、型定義一覧に表示されます。
- Doxygen では、"@typedef" コマンドに相当します。

4.3.23. @union コマンド

共用体名を定義します。

構文

```
@union 【共用体名】 <共用体和名>
```

記述例

```
/**!  
@union CommonData データ用共用体  
*/
```

出力例

共用体

- CommonData : データ用共用体

備考

- 定義された共用体は、共用体一覧に表示されます。

4.3.24. @varコマンド

変数名を定義します。

構文

```
@var 【変数名】 <変数和名>
```

記述例

```
int id; /**!< @var id 識別子 */
```

出力例

変数
id
識別子

備考

- 「記述例」はインラインコメントを使用した場合のものです。
- 定義された変数名は、変数一覧に表示されます。

4.3.25. @warningコマンド

警告事項を定義します。

構文

@warning 【警告事項】

記述例

```
/**!  
@warning 実行には管理者権限が必要です。  
*/
```

出力例



実行には管理者権限が必要です。

備考

- AsciiDoc のデフォルトアイコンを使用します。

5. Doxygen で使用する AsciiDoc 記法

Doxygen のコメントブロック内では、Doxygen コマンドとともに AsciiDoc 記法による記述もできます。

本章では、Doxygen で利用頻度の高い AsciiDoc 記法について説明します。

AsciiDoc のすべての機能を解説するものではありません。

詳細については、Asciidoctor の公式ドキュメントを参照してください。

5.1. 段落と改行

AsciiDoc では、空行によって段落を区切ります。

また、同一段落内で改行する場合は、行末に半角スペースと + 記号を付加します。

記述例

ここは1つめの段落です。

ここは2つめの段落の1行目です。 +

ここは2つめの段落の2行目です。

出力例

ここは1つめの段落です。

ここは2つめの段落の1行目です。

ここは2つめの段落の2行目です。

この "+" は、Doxygen の "@n" コマンドに相当します。

5.2. 文字の装飾

AsciiDoc による文字装飾の例を表 5.1 に示します。

表 5.1 文字装飾の例

AsciiDoc 記法	表示内容
太字	太字
<i>__shatai__</i>	shatai (斜体)
<code>`monospaced font`</code>	monospaced font (等幅フォント)
##強調表示##	強調表示
[.red]##赤色文字##	赤色文字

AsciiDoc 記法	表示内容
[.underline]##下線##	<u>下線</u>
[.line-through]## 取り消し線##	取り消し線

装飾は必要最小限とし、読みやすさを優先してください。

5.3. 見出し

見出しは "=" の数によって階層を表します。

なお、"=" はドキュメントタイトルを定義するものであり、本書では不使用とします。

記述例

```
== 見出しレベル 1 (章)

=== 見出しレベル 2 (節)

==== 見出しレベル 3 (項)
```

出力例

```
1. 見出しレベル 1 (章)
  1.1. 見出しレベル 2 (節)
    1.1.1. 見出しレベル 3 (項)
```

5.4. 箇条書き

箇条書きは "*" を使用します。

記述例

```
* 項目1
** 項目1-1
*** 項目1-1-1
```

出力例

```
• 項目1
```

- 項目1-1
 - 項目1-1-1

番号付き箇条書きは"."を使用します。
ただし、"."（ピリオド 1つ）は、見出しレベル 1の表記と重複するため、".. "（ピリオド 2つ）から使用します。

記述例

```
.. 手順1
... 手順1-1
.... 手順1-1-1
```

出力例

```
a. 手順1
  i. 手順1-1
    A. 手順1-1-1
```

5.5. ラベル付きリスト

ラベル付きリストは、用語と説明を対にして記述する場合に使用します。

記述例

```
入力::
入力ファイルを指定します。

出力::
出力ディレクトリを指定します。
```

出力例

```
入力
  入力ファイルを指定します。

出力
  出力ディレクトリを指定します。
```

5.6. ブロック

AsciiDoc には、用途に応じて複数のブロックがあります。
本書では、その一例を示します。

記述例

```
====  
ここはブロックです。 +  
複数行をまとめて1つのブロックとします。  
=====
```

出力例

ここはブロックです。
複数行をまとめて1つのブロックとします。

ブロック内で改行するには、行末に "+" を付加します。

5.7. アドモニション

注意事項などを強調する場合はアドモニションを使用します。

記述例

NOTE: メモです。

TIP: ヒントです。

IMPORTANT: 重要事項です。

WARNING: 注意してください。

出力例

	メモです。
	ヒントです。
	重要事項です。



注意してください。

5.8. 画像

画像は、"@figure" コマンドと "image" マクロを組み合わせで記述します。

記述例

```
[[fig.5.1]] ①
@figure asciidoctor 公式ロゴ ②
image:../images/asciidoctor_icon.png[width="20%", align="center"] ③
```

①：インラインアンカー

本文内で図を参照するための識別子です。
本書では、"fig.【章番号】.【章内連番】" の形式で記述します。

②："@figure" コマンド

図のタイトルを記述します。

③："image" マクロ

"image::"（コロンは2つ）に続けて、画像ファイル名と表示オプションを指定します。

画像ファイル名

拡張子を含む画像ファイル名を絶対パス、または相対パスで指定します。

width

"width" 属性は、画像の幅を "px" または "%" で指定します。

align

"align" 属性は、画像の表示位置を指定します。
"left"、"center"、"right" のいずれかを指定します。

なお、本文内で図を参照する場合は、"<<fig.x.y>>" と記述します。

出力例



図 5.1 asciidoctor 公式ロゴ

5.9. テーブル

テーブルは、"@table" コマンドとテーブルブロックを組み合わせて記述します。

記述例

```
[[tbl.5.1]] ①
@table コマンドによるテーブルの例 ②
[cols="1,2", options="header", width="90%", role="center"] ③
|=== ④

^.^|項目 ⑤
^.^|説明

|入力 ⑥
|入力ファイル

|出力
|出力ディレクトリ
|=== ⑦
```

①：インラインアンカー

本文内で表を参照するための識別子です。
本書では、"tbl.【章番号】.【章内連番】" の形式で記述します。

②："@table" コマンド

表のタイトルを記述します。

③：テーブル属性

テーブルの列数と幅、ヘッダー、テーブルの幅、および表示位置に関する属性を指定します。

cols

"cols" 属性は、テーブルの列数と列幅を同時に指定します。

```
"【1列目の幅】,<2列目の幅>,...,<n列目の幅>"
```

カンマで区切った数字の数だけ列が作成され、各数字は、列幅の相対的な割合を表します。
例えば、"1,2" は、列数が "2"、1列目と2列目の幅が "1:2" になることを示します。

options

`header` を指定すると、先頭行をテーブルヘッダーとして扱います。

width

"width" 属性は、テーブル全体の幅を`%`で指定します。

role

"role" 属性は、テーブルの表示位置を調整するためのロールを指定します。
"left"、"center"、"right" のいずれかを指定します。

④：テーブル開始宣言

テーブルブロックの開始を示します。

⑤：テーブルヘッダー

ヘッダーとなる項目を記述します。
なお、"^.^"は、セル内の文字を横位置を中央揃えに、縦位置も中央揃えにすることを示します。

⑥：行データ

各セルの内容を "|" に続けて記述します。
この例では、2行で1つのデータ行となります。

⑦：テーブル終了宣言

テーブルブロックの終了を示します。

なお、本文内でテーブルを参照する場合は、 "<<tbl.5.1>>" と記述します。

出力例

表 5.2 @table コマンドによるテーブルの例

項目	説明
入力	入力ファイル
出力	出力ディレクトリ

5.10. ソースコード

ソースコードを記述する場合は、ソースコードブロックを使用します。

記述例

```
[source, python] ①
----
def add(a: int, b: int) -> int:
    return a + b
----
```

①：ソースブロック属性

言語識別子に関する属性を指定します。

python

言語識別子。

言語識別子を指定すると、ソースコードが構文ハイライト表示になります。
使用できるプログラミング言語は表 5.3 を参照のこと。

出力例

```
def add(a: int, b: int) -> int:
    return a + b
```

表 5.3 にソースコードブロックに使用できるプログラミング言語と言語識別子を示します。

主な言語名

表 5.3 プログラミング言語と言語識別子

プログラミング言語	言語識別子
Python	python
C	c
C++	cpp
Java	java
JavaScript	javascript
TypeScript	typescript
VBA	vb
DOS	dos, bat
テキスト	text, txt

また、ソースコードをファイルから読み込む場合は、"include" マクロを使用します。



Doxygen は、"include" マクロを展開しません。
"include" マクロは、生成した AsciiDoc ファイルを HTML/PDF に変換する際に行われます。

記述例

```
[source, c, linenums, start=1,6] ①
----
include:../src/sample.c[] ②
----
```

①：ソースブロック属性

言語識別子、行番号表示オプションなどに関する属性を指定します。

c

言語識別子。
言語識別子を指定すると、ソースコードが構文ハイライト表示になります。
使用できるプログラミング言語は表 5.3 を参照のこと。

linenums

行番号表示オプション。
ソースコードの左側に行番号が表示されます。

start

表示範囲オプション。表示する行番号を指定します。
表示開始行番号のみを指定した場合は、指定された行番号から最終行までが表示されます。
表示開始行番号と表示終了行番号を指定した場合は、指定された範囲が表示されます。

②: "include" マクロ

"include::" (コロンは2つ) に続けて、ソースファイル名を指定します。

ソースファイル名

拡張子を含むソースファイル名を絶対パス、または相対パスで指定します。

出力例

```
1 #include <stdio.h>
2
3 int main(void) {
4     printf("Hello, World!\n");
5     return 0;
6 }
```

5.11. コメント

コメントは、成果物へ出力したくないメモを記述する場合に使用します。

記述例

```
// この行はコメントです。
```

出力例

また、複数行のコメントは、

```
////
ここに記述した内容は
すべてコメントとなります。
////
```

と記述します。

5.12. DoxygenコマンドとAsciiDoc記法の対比

Doxygenは、すべてのDoxygenコマンドを独自コマンドとして実装しているわけではありません。文字の装飾、改行、箇条書き、見出し、リンクなど、AsciiDocの標準記法で表現できるものについては、コメントブロック内にAsciiDocを直接記述します。主なDoxygenコマンドとAsciiDoc記法の対比を以下に示します。

表 5.4 Doxygen コマンドとの対比

Doxygen コマンド	Doxygenでの記述	説明
@n	行末の ` + `	段落内で改行します。
@b	**文字列**	文字列を太字にします。
@e、@em	<i>__文字列__</i>	文字列を斜体にします。
@c、@p	`文字列`	文字列を等幅フォントで表示します。
@li	* 文字列	番号なしの箇条書きを記述します。
@code ~@endcode	[[source,言語名]]と "----" ブロック	ソースコードを記述します。
@verbatim ~@endverbatim	"...." ブロック	入力した文字列を整形せずに表示します。
@section 、@subsection	"==", "===", "===="	"=" の数により見出しの階層を指定します。
@ref	<<アンカーID,表示文字列>>	文書内の別の位置を参照します。
@link ~@endlink	URL [表示文字列]	外部ページへのリンクを記述します。
@image	"@figure" と "image::ファイル名[]"	タイトル付きの画像を挿入します。



上表は、DoxygenコマンドをDoxygenがAsciiDoc記法へ自動変換することを示すものではありません。Doxygenでは、対応するAsciiDoc記法をコメントブロック内へ直接記述します。